

HANDOVER — «L'attesa del coltello» (Sala I)

Matteo Proietti

Documento di passaggio per il team creativo. Nessuna credenziale inclusa: chiavi API, account e repository vanno configurati dal team (vedi §8).

INDICE

- | | |
|-------------------------------|---|
| 1. Contesto | 6. Strumenti / tool esterni |
| 2. Output prodotti | 7. Come modificare / rilavorare l'output |
| 3. Mappa del repository | 8. Come usare Claude Code (onboarding team) + setup strumenti |
| 4. Metodo | 9. Stato e cose aperte |
| 5. Skills usate (Claude Code) | |

1. Contesto

Progetto accademico IED Roma, corso **Prompt Thinking** (Andrea Colamedici). L'esercizio: **costruire un artista contemporaneo finzionale** + il suo **corpus** + un **dispositivo curatoriale** (una sala), nella tradizione degli artisti costruiti (Reena Spaulings, The Atlas Group, Goldin+Senneby, Nat Tate). Mostra collettiva all'**ex Mattatoio di Testaccio, La Pelanda** (14 nov 2026 – 28 feb 2027).

L'artista — Matteo Proietti (Roma, 2001): poeta. Frattura: perde il più importante concorso di poesia d'Italia contro una poesia scritta interamente da un'IA → smette di scrivere e «smette di finire». Usa l'IA ma la **arresta prima che risolva**.

La sala — «L'attesa del coltello» (Sala I): la soglia della mostra, l'unica in cui «il coltello non cade». Coltello = IA, animale/maiale = uomo (il mattatoio). Tesi: **«tutto ciò che non è ancora stato tagliato»**.

Principio guida (da non rompere mai): *l'artista non finisce le opere; la documentazione è impeccabile*. Le opere sono stati intermedi, arrestati; il catalogo è perfetto.

Filtro anti-«Black Mirror» (regola estetica): stare sul *prima* (non l'esito), creare uno *stato* non una *storia*, l'umano come soggetto (non la tecnologia), ambiguità non morale, presente non futuro distopico, registro di sospensione. Vedi `CONCEPT.md §5`.

2. Output prodotti

- **Sito-mostra interattivo** (Vite + Three.js). Deployment attuale: <https://site-seven-rouge-36.vercel.app> (da rideployare sul proprio account, §8).
- **Corpus di 12 opere:** 5 immagini «arrestate» (op1, 4, 5, 8, 11), audio generativo (op2 lamento, op9 respiro), formula olfattiva (op3), video (op7, Veo), testo/generativo (op10, op12), archivio (op6).
- **Catalogo d'esame:** `catalogo/catalogo.pdf` (31 pp, 6 sezioni) + sorgente HTML/CSS.
- **Allestimento:** pianta 95 m² (`pianta.svg`), 2 render, scheda sensoriale.
- **Comunicazione:** comunicato, 2 post, invito, recensione su rivista fittizia.
- **Pacchetto di consegna:** `CONSEGNA/` (+ zip).
- **Documenti di metodo:** `CONCEPT.md`, `PLAN.md`, `STATUS.md`, `DECISIONI.md`, `SPESE.md`, `FINAL_REPORT.md`.

3. Mappa del repository (dove sta cosa)

/site	Sito (Vite + Three.js vanilla)
src/data/opere.js	← LE 12 OPERE: dati + scheda a 6 voci (fonte per sito e catalogo)
src/ui/	corpus.js (griglia), plate.js (provini), opera-overlay.js (vista opera), intro.js (loader), cursor.js
src/scenes/	denoise.* (home), interpolate.* (op7), ultimariga.js (op12)
src/audio/	respiro.js (op9), lamento.js (op2)
public/opere/	immagini, video (op07-veo.mp4), audio
src/styles/	design-tokens.css (palette/typo), base.css, layout.css
/assets/immagini	opere sorgente + gen.py (Stability) + arresto.py (post «arresto») + generation-log.md
/assets/video	op7_veo.mp4 + veo_gen.py (Veo)
/output/testi	nome, biografia-critica, statement, versi-abbandonati, testi-di-parete (.md)
/output/schede.md	le 12 schede a 6 voci
/output/identita.md	sistema visivo (palette HEX, tipografia, motion)
/output/allestimento	pianta.svg, render/, scheda-sensoriale.md, allestimento.md
/catalogo	catalogo.html, catalogo.css, fonts/ (self-hostati), img/, catalogo.pdf
/comms	comunicato, social, invito, recensione fittizia
.env	chiavi API (NON in git) – da creare (§8)

4. Metodo (come è stato costruito)

1. **Brainstorming strutturato** → diagnosi del rischio «Black Mirror» → **checklist anti-BM** (filtro su ogni opera).
2. **Posizione dell'artista** prima dello stile → 2–3 direzioni concettuali → **pressure test** a 5 filtri → concept bloccato (**CONCEPT.md**).
3. **Grammatica del corpus** (tratti ricorrenti che rendono coerenti le 12 opere): arresto prima della risoluzione, palette osso/grigio-fiato/rosso-trattenuto, grana di diffusione, ~35,9 °C, verso sepolto come «provenienza», titoli al grado incompiuto.
4. **Esecuzione a ondate con agenti paralleli**: un agente per dominio indipendente (scrittura · immagini · sito · audio · allestimento · comunicazione · catalogo), con review e checkpoint git

frequenti su un branch dedicato + PR finale.

- 5. Regola tecnica chiave per le immagini:** il look «arrestato» NON si spera dal modello — si garantisce in post-processing (`arresto.py`). Il modello dà solo la forma base.

5. Skills usate (Claude Code)

- **Processo:** `superpowers:brainstorming` , `superpowers:writing-plans` , `superpowers:executing-plans` , `superpowers:dispatching-parallel-agents` .
- **Front-end / visual:** `frontend-design` , `web-design-3d` , `web-artifacts-builder` , `algorithmic-art` , `theme-factory` , `brand-guidelines` , `canvas-design` .
- **Documenti:** `pdf` .
- **Copy / comms:** `copywriting` , `marketing-ideas` .
- **QA:** `webapp-testing` .

Nota: una skill di finalizzazione autonoma («goal») non era disponibile nell'ambiente; sostituita con `executing-plans` + un loop di checkpoint.

6. Strumenti / tool esterni

- **Immagini:** Stability AI — modello **SD3.5 Medium** via API REST. Script `assets/immagini/gen.py` (usa `curl` con User-Agent browser per aggirare un blocco Cloudflare). Post-processing in Python (Pillow + numpy): `assets/immagini/arresto.py` .
- **Video:** Google **Veo 3.1** (`veo-3.1-generate-preview`) via Gemini API (operazione long-running + polling). Script `assets/video/veo_gen.py` .
- **Audio:** Web Audio API (codice nativo in `site/src/audio/`) + **ElevenLabs** (una voce sussurrata in op2).
- **Sito:** **Vite** + **Three.js** / **WebGL** (vanilla, nessun framework).
- **Deploy:** **Vercel** (hosting statico).
- **Render PDF:** **Chrome headless** (stampa HTML→PDF).

7. Come modificare / rilavorare l'output

Testi (bio, statement, versi, schede)

Modifica i `.md` in `output/testi/` e `output/schede.md` , e i campi corrispondenti in `site/src/data/opere.js` . Tieni **distinte le due voci**: curatore (3^a persona, registro da catalogo) e artista (1^a persona, mai moralizzante).

Opere-immagine (op1, 4, 5, 8, 11)

Rigenera la base con `gen.py` (cambia `prompt`, `seed`, `steps`, `cfg`), poi applica `arresto.py` (i parametri usati sono in `generation-log.md`). Il look «arrestato» si regola tutto in `arresto.py`: gradient-map alla palette, rumore/grana, dissolvenza dei bordi, forma risolta ~40–55 %, rosso trattenuto in modalità `tint`. Esporta `.webp` e mettilo in `site/public/opere/` con il nome atteso (es. `01-studio-corpo.webp`, `op11-calco-del-fiato.webp`).

Video (op7)

Rigenera con `veo_gen.py` (cambia il prompt). Sostituisci `site/public/opere/op07-veo.mp4` (+ `op07-veo-poster.jpg`). Il sito monta il video se il file esiste, altrimenti usa lo shader di interpolazione (fallback automatico).

Opere vive (audio / shader)

Codice in `site/src/audio/` (respiro, lamento) e `site/src/scenes/` (interpolate, ultimariga). Espongono `start()` / `stop()` / `destroy()` e si fermano fuori vista.

Sito

`cd site && npm install && npm run dev` (sviluppo) · `npm run build` (produzione → `site/dist`). Le opere si aggiungono/modificano in `src/data/opere.js` (campi: `plate`, `live`, le 6 voci della scheda, `verso`). Palette/tipografia in `src/styles/design-tokens.css` e `output/identita.md`.

Catalogo

Modifica `catalogo/catalogo.html` e `catalogo/catalogo.css` (i font sono self-hostati in `catalogo/fonts/`). Rigenera il PDF con Chrome headless:

```
"<chrome>" --headless --no-pdf-header-footer \  
--print-to-pdf=catalogo.pdf catalogo.html
```

Allestimento

`pianta.svg` è vettoriale ed editabile; i render si rigenerano con `gen.py`.

Comunicazione

Modifica i `.md` in `comms/`.

Pubblicazione

Dalla cartella `site/`: `vercel deploy --prod` (serve un account Vercel + token). Output statico già pronto in `site/dist`.

8. Come usare Claude Code (onboarding team) + setup strumenti

Claude Code è un assistente AI da terminale: legge e scrive i file del progetto, esegue comandi, genera asset tramite script e può orchestrare più «agenti» in parallelo. Si lavora parlandogli in linguaggio naturale dentro la cartella del progetto.

Skills (capacità riutilizzabili)

Le skills (es. `frontend-design`, `pdf`) si invocano scrivendo `/nome-skill` o chiedendo a Claude di usarle. Si installano come **plugin**: dal gestore plugin (`/plugin`) si aggiunge un *marketplace* e si installano i plugin che contengono le skill. In alternativa, una skill è una cartella con un file `SKILL.md` (+ risorse) da mettere in `.claude/skills/` (progetto) o `~/.claude/skills/` (utente).

Tool esterni via MCP

Model Context Protocol — es. Figma, Canva, Vercel, Microsoft 365. Si aggiungono come *server MCP* nella configurazione di Claude Code (comando `claude mcp add ...` oppure dai settings). I connettori con login (OAuth) si attivano tramite il flusso di autenticazione integrato (Claude mostra un link da aprire nel browser).

Chiavi API dei modelli generativi (Stability, Gemini/Veo, ElevenLabs)

1. Crea un account su ciascun servizio e ottieni la API key.
2. Crea un file `.env` nella root del progetto con righe `NOME_CHIAVE=valore` (es. `STABILITY_API_KEY=...`, `GEMINI_API_KEY=...`, `ELEVENLABS_API_KEY=...`, `VERCEL_TOKEN=...`).
3. **Non committare mai** `.env` (è già in `.gitignore`). Gli script `gen.py` / `veo_gen.py` leggono le chiavi da lì.

Attenzione shell: non «sorgentare» il `.env` in zsh; leggi i valori con
`grep '^NOME=' .env | cut -d= -f2-`.

Flusso di lavoro consigliato

(Lo stesso usato in questo progetto): chiedi a Claude di *brainstormare* → far scrivere un *piano* → eseguire con *agenti paralleli* (un agente per dominio indipendente) → *commit frequenti su un branch* → *PR*. Tieni un file `STATUS.md` come tracker per poter riprendere in qualsiasi momento.

9. Stato e cose aperte

- **Wave UI in corso:** effetti sito aggiuntivi (transizione d'entrata «nella sala», navigazione tra le opere nell'overlay, card 3D fluttuanti).
- **Non realizzato (extra opzionale):** «libro-catalogo non tagliato» fisico (con coltello di carta).

- **Sicurezza:** il team usi le proprie chiavi nel `.env` locale; rigenerare eventuali chiavi condivise.
- Avanzamento dettagliato e decisioni: `STATUS.md`, `DECISIONI.md`, `SPESE.md`.